

Nama : Hakashi Yumiko Masfat
NIM : J0403251023
Kelas : TPL A2

PRAKTIKUM 11

1. Membuat Adjacency Matrix

```
#####  
# NAMA : Hakashi Yumiko Masfat  
# NIM : J0403251023  
# KELAS : TPL A2  
#####  
  
V = 4  
matrix = [  
    [0,1,1,0],  
    [1,0,1,0],  
    [1,1,0,1],  
    [0,0,1,0]  
]  
print("Adjacency Matrix:")  
for row in matrix:  
    print(row)
```

```
iYumiko/Praktikum11/adjacencyMatrix.py  
Adjacency Matrix:  
[0, 1, 1, 0]  
[1, 0, 1, 0]  
[1, 1, 0, 1]  
[0, 0, 1, 0]  
PS C:\Users\Hakas\OneDrive\Documents\ASD_J0403251023_  
raktikum11>
```

- baris 0 menunjukkan node 0 terhubung dengan node 1 dan node 2
- baris 1 menunjukkan node 1 terhubung dengan node 0, node 1, dan node 2
- baris 2 menunjukkan node 2 terhubung dengan node 0, node 1, dan node 3
- baris 3 menunjukkan node 3 terhubung dengan node 2

2. Membuat Adjacency List

```
#####  
# NAMA : Hakashi Yumiko Masfat  
# NIM : J0403251023  
# KELAS : TPL A2  
#####  
  
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D'],  
    'C': ['A', 'D'],  
    'D': ['B', 'C']  
}  
print("Adjacency List:")  
for node, neighbors in graph.items():  
    print(f"{node}: {(neighbors)}")
```

```
iYumiko/Praktikum11/adjacencyList.py  
Adjacency List:  
A: ['B', 'C']  
B: ['A', 'D']  
C: ['A', 'D']  
D: ['B', 'C']  
PS C:\Users\Hakas\OneDrive\Documents\ASD_J04  
raktikum11>
```

- A terhubung ke B dan C.
- B terhubung ke A dan D.
- C terhubung ke A dan D.
- D terhubung ke B and C.

3. Konversi Matrix ke List

```
#####
# NAMA : Hakashi Yumiko Masfat
# NIM : J0403251023
# KELAS : TPL A2
#####

matrix = [
    [0,1,1,0],
    [1,0,1,0],
    [1,1,0,1],
    [0,0,1,0]
]

adj_list = {}
for i in range(len(matrix)):
    neighbors = []
    for j in range(len(matrix[i])):
        if matrix[i][j] == 1:
            neighbors.append(j)
    adj_list[i] = neighbors

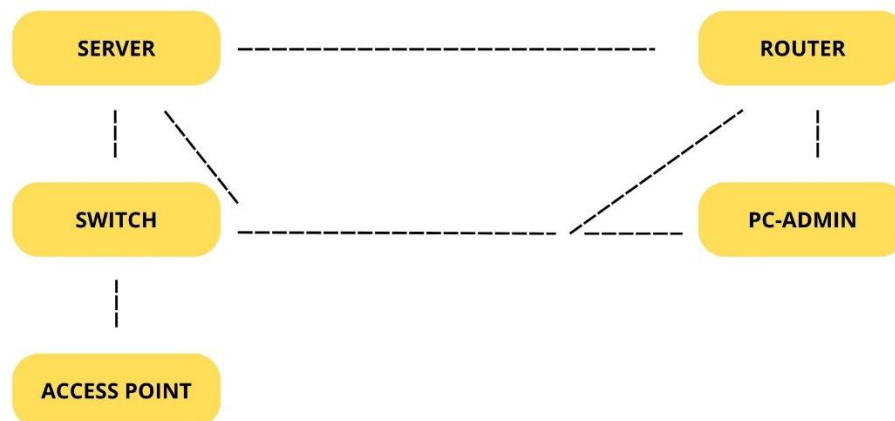
print("Hasil Adjacency List:")
for node, neighbors in adj_list.items():
    print(f"{node}: {neighbors}")
```

```
iYumiko/Praktikum11/konversiMatriksList.py
Hasil Adjacency List:
0: [1, 2]
1: [0, 2]
2: [0, 1, 3]
3: [2]
PS C:\Users\Hakas\OneDrive\Documents\ASD_J0403251023\Praktikum11>
```

- Baris 0: Angka 1 ada di indeks 1 dan 2 >>> 0: 1, 2.
- Baris 2: Angka 1 ada di indeks 0, 1, dan 3 >>> 2: 0, 1, 3.
- Baris 1: Angka 1 ada di indeks 0 dan 2 >>> 1: 0, 2.
- Baris 3: Angka 1 ada di indeks 2 >>> 3: 2.

4. Studi Kasus

- Studi Kasus: Jaringan Komputer.
- Topologi: Star-Mesh Hybrid.



Detail hubungan untuk gambar:

- Router Utama ↔ Server
- Router Utama ↔ Switch
- Router Utama ↔ PC-Admin
- Switch ↔ Server
- Switch ↔ PC-Admin
- Switch ↔ Access Point
- PC-Admin ↔ Server

```

# 1. Representasi Adjacency List
graph = {
    "Router Utama": ["Server", "Switch", "PC-Admin"],
    "Server": ["Router Utama", "Switch", "PC-Admin"],
    "Switch": ["Router Utama", "Server", "PC-Admin", "Access Point"],
    "PC-Admin": ["Router Utama", "Server", "Switch"],
    "Access Point": ["Switch"]
}

# 2. Representasi Adjacency Matrix
nodes = ["Router Utama", "Server", "Switch", "PC-Admin", "Access Point"]
matrix = [
    [0, 1, 1, 1, 0], # Router Utama
    [1, 0, 1, 1, 0], # Server
    [1, 1, 0, 1, 1], # Switch
    [1, 1, 1, 0, 0], # PC-Admin
    [0, 0, 1, 0, 0] # Access Point
]

print("=== REPRESENTASI GRAPH JARINGAN KOMPUTER ===")

print("\n1. Adjacency List:")
for node, neighbors in graph.items():
    print(f"{node}: {neighbors}")

print("\n2. Adjacency Matrix:")
print("      ", end="")
for n in range(len(nodes)):
    print(f"[{n}] ", end="")
print()
for i in range(len(matrix)):
    print(f"Node [{i}] {nodes[i]:<12}: {matrix[i]}")

print("\n3. Daftar Nama Node (Vertex):")
for i, node in enumerate(nodes):
    print(f"Indeks {i}: {node}")

print("\n4. Hubungan Antar Node (Edges):")
for node, neighbors in graph.items():
    for neighbor in neighbors:
        if node < neighbor:
            print(f"- {node} terhubung dengan {neighbor}")

```

1. Jenis graph: graph ini adalah Undirected Graph karena hubungan antar perangkat jaringan fisik bersifat dua arah.
2. Alasan memilih edge: tujuh edge dipilih untuk menjamin redundansi jalur data pada server dan efisiensi distribusi akses melalui switch.
3. Representasi yang lebih cocok: Adjacency List lebih cocok karena lebih hemat memori dan memudahkan proses penambahan node baru di masa depan.
4. Analisis kerapatan graph: graph ini termasuk Sparse Graph karena jumlah koneksi yang digunakan hanya 7 dari 10 total kemungkinan hubungan maksimal.

Kesimpulan: Struktur graph sangat efektif untuk memetakan infrastruktur TI secara visual dan programatis.